END-TO-END FMCG RETAIL SALES ANALYTICS PROJECT POWERED BY EXCEL | SQL | PYTHON | POWER BI

A complete data analytics project covering data cleaning, transformation, analysis, and dashboard creation for FMCG retail sales.

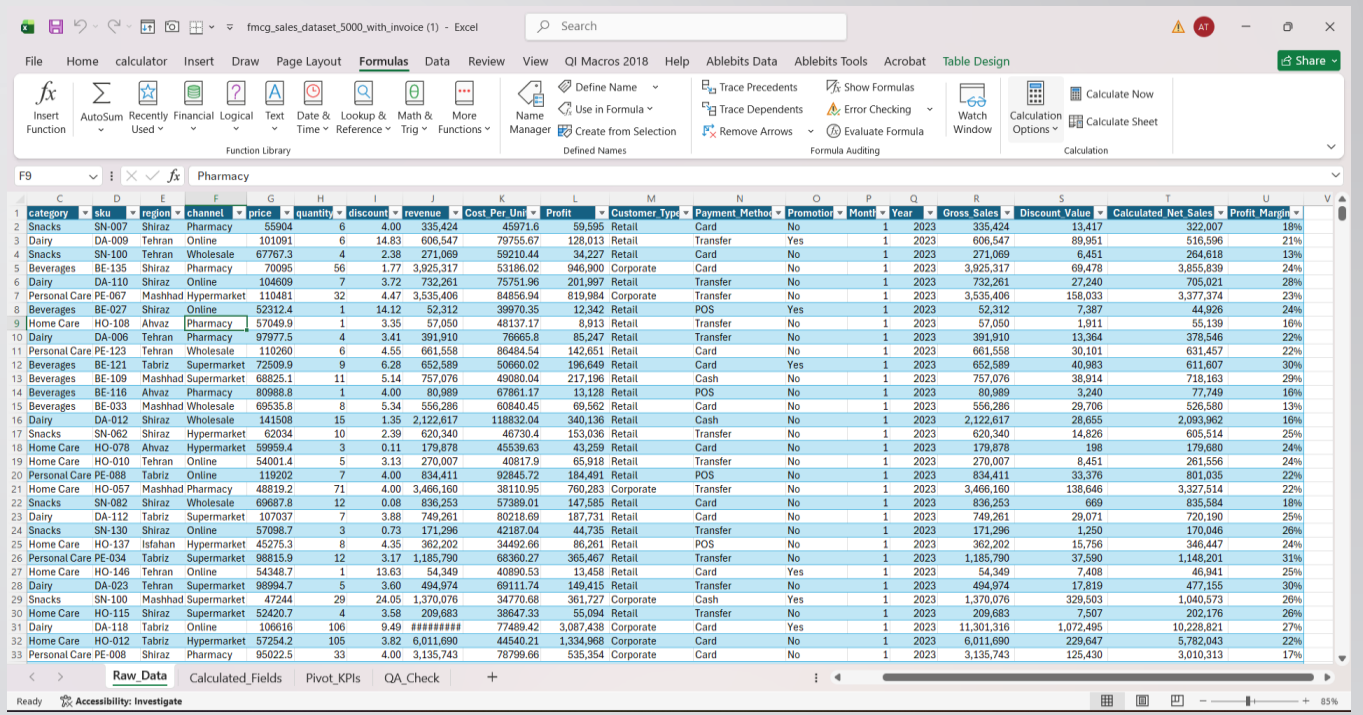
Work Flow: Excel → SQL → Python → Power BI → Insights

Prepared by: Ali Khanbeigi

1. Excel Stage:

1. Data Review & Initial Cleaning

Field	Description
Project Domain	FMCG / Retail Sales
Objective	Analyze product, region, and channel performance
Dataset Source	CSV
Tools (Pipeline)	Excel ⇢ SQL ⇢ Python ⇢ Power BI
Record Count	~5K transactions
Time Range	Apr 2026 – Mar 2026
sheets	Rowdata-Calculated fields-Pivot KPI- QA Check



1. Excel Stage:

2. Calculated Fields: Adding some crucial fields and KPIs to the table

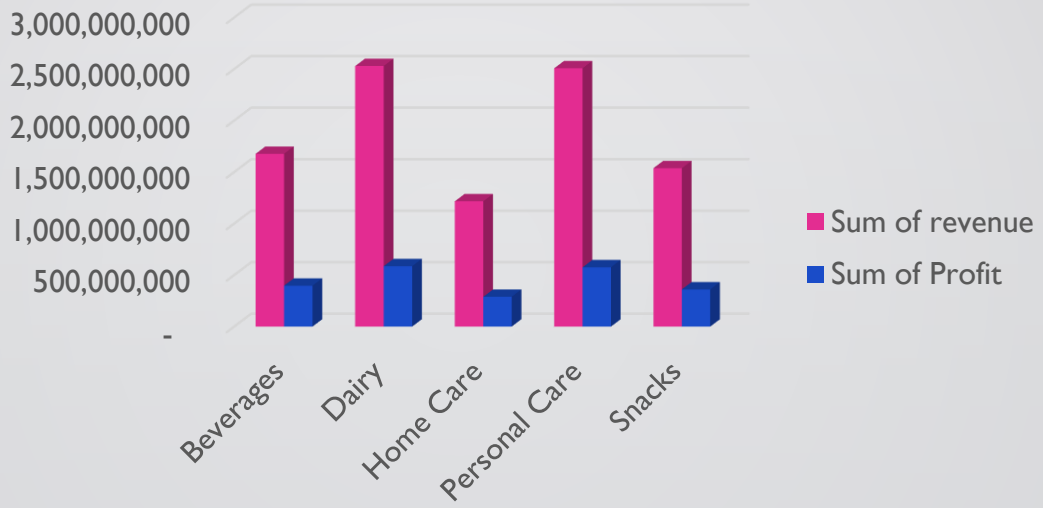
Field Name	Formula Example	Description
Gross_Sales	=[@price]*[@quantity]	Total revenue before discount
Discount_Value	=[@Gross_Sales]*([@discount]/100)	Monetary value of discount
Calculated_Net_Sales	=[@Gross_Sales]-[@Discount_Value]	Net sales after discount
Profit_Margin	=[@profit]/[@revenue]	Margin ratio

1. Excel Stage:

3. Pivot KPI: This pivot table provides an initial summary of sales and profitability by product category, helping identify top-performing segments before moving to deeper analysis in another stages. For example:

category	Sum of revenue	Sum of Profit
Beverages	1,676,337,233	396,218,459
Dairy	2,529,114,293	586,011,200
Home Care	1,216,291,730	288,819,856
Personal Care	2,507,200,832	575,251,609
Snacks	1,537,237,687	360,499,833
Grand Total	9,466,181,774	2,206,800,957

Pivot table



Bar chart

Slicer

This pivot table calculates total revenue and profit across product categories, allowing quick comparison of financial performance between segments. There are some other pivot tables like: sum profit and sum revenue by region, channel, year and month, promotion,....

1. Excel Stage:

3. QA Check : Performed data QA checks to ensure completeness, accuracy, and consistency before loading into Power BI.

Revenue Check	Mismatch	Formula
Negative Profit	OK	=IF(COUNTIF(SALES_DATA[Profit], "<0")=0, "OK", "Found Negative Profit")
Max Profit	5,150,518	=MAX(SALES_DATA[Profit])
Min Profit	6,117	=MIN(SALES_DATA[Profit])
Min Revenue	38,398	=MIN(SALES_DATA[revenue])
Max Revenue	18,346,547	=MAX(SALES_DATA[revenue])
Total Orders	5000	-

QA Table

- Validated key columns, checked for duplicates, and confirmed category mappings were correct.
- Reviewed calculated fields and pivot outputs to verify correctness of revenue and profit logic.
- Ensured no missing values and standardized formatting across all source tables.

2. SQL Stage(SQL Server):

1. Data Import & Table Creation

Created a structured SQL table and imported raw FMCG sales data using BULK INSERT to enable efficient querying and validation before analysis.

CREATE TABLE.sql -...r (ASUS\ASUS (70))

```sql
DROP TABLE IF EXISTS fmcg_sales_dataset_new;
CREATE TABLE fmcg_sales_dataset_new (
    Invoice_ID NVARCHAR(50),
    [date] DATE,
    category NVARCHAR(100),
    sku NVARCHAR(50),
    region NVARCHAR(50),
    channel NVARCHAR(50),
    price DECIMAL(18,4),
    quantity INT,
    discount DECIMAL(10,4),
    revenue DECIMAL(18,4),
    Cost_Per_Unit DECIMAL(18,4),
    Profit DECIMAL(18,4),
    Customer_Type NVARCHAR(50),
    Payment_Method NVARCHAR(50),
    Promotion NVARCHAR(50),
    Month INT,
    Year INT,
    Gross_Sales DECIMAL(18,4),
    Discount_Value DECIMAL(18,4),
    Calculated_Net_Sales DECIMAL(18,4)
);


BULK INSERT fmcg_sales_dataset_new
FROM 'C:\Temp\fmcg.csv'
WITH (
    FIRSTROW = 2,
    FIELDTERMINATOR = ',',
    ROWTERMINATOR = '\n',
    TABLOCK
);
SELECT COUNT(*) FROM fmcg_sales_dataset_new;

SELECT TOP 10 * FROM fmcg_sales_dataset_new;
```

Results | Messages

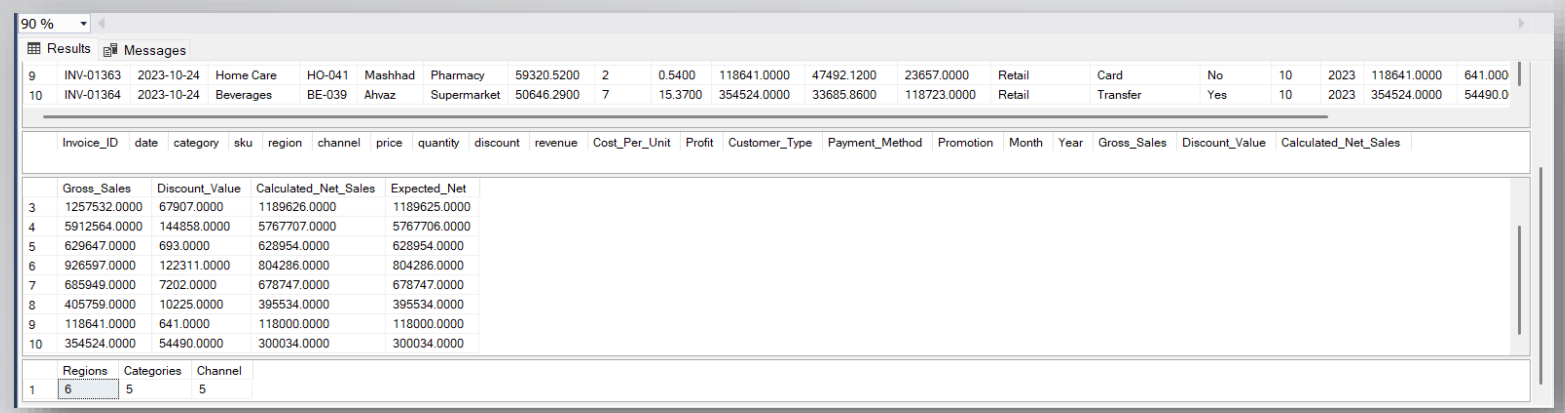
	(No column name)
1	5000

	Invoice_ID	date	category	sku	region	channel	price	quantity	discount	revenue	Cost_Per_Unit	Profit	Customer_Type	Payment_Method	Promotion	Month	Year	Gross_Sales	Discount_Value
1	INV-01355	2023-10-22	Personal Care	PE-077	Isfahan	Hypermarket	110164.0100	20	3.0400	2203280.0000	80399.9600	595281.0000	Corporate	Transfer	No	10	2023	2203280.0000	66980.0000
2	INV-01356	2023-10-22	Beverages	BE-096	Ahvaz	Pharmacy	55602.0400	91	22.2100	5059786.0000	46555.3200	823252.0000	Corporate	Transfer	Yes	10	2023	5059786.0000	1123778.0000
3	INV-01357	2023-10-23	Dairy	DA-117	Ahvaz	Hypermarket	114321.1200	11	5.4000	1257532.0000	79931.2700	378288.0000	Retail	Card	No	10	2023	1257532.0000	67907.0000
4	INV-01358	2023-10-23	Personal Care	PE-069	Ahvaz	Hypermarket	103729.2000	57	2.4500	5912564.0000	76125.5400	1573409.0000	Corporate	Card	No	10	2023	5912564.0000	144858.0000
5	INV-01359	2023-10-24	Snacks	SN-150	Mashhad	Supermarket	57240.6000	11	0.1100	629647.0000	38751.4000	203381.0000	Retail	Card	No	10	2023	629647.0000	693.0000
6	INV-01360	2023-10-24	Personal Care	PE-044	Mashhad	Pharmacy	102955.1700	9	13.2000	926597.0000	82225.1500	186570.0000	Retail	Card	Yes	10	2023	926597.0000	122311.0000
7	INV-01361	2023-10-24	Snacks	SN-063	Isfahan	Supermarket	68594.9400	10	1.0500	685949.0000	45314.0800	232809.0000	Retail	POS	No	10	2023	685949.0000	7202.0000
8	INV-01362	2023-10-24	Home Care	HO-061	Tabriz	Pharmacy	57965.5200	7	2.5200	405759.0000	45738.1700	85591.0000	Retail	Transfer	No	10	2023	405759.0000	10225.0000
9	INV-01363	2023-10-24	Home Care	HO-041	Mashhad	Pharmacy	59320.5200	2	0.5400	118641.0000	47492.1200	23657.0000	Retail	Card	No	10	2023	118641.0000	641.0000
10	INV-01364	2023-10-24	Beverages	BE-039	Ahvaz	Supermarket	50646.2900	7	15.3700	354524.0000	33685.8600	118723.0000	Retail	Transfer	Yes	10	2023	354524.0000	54490.0000

2. SQL Stage(SQL Server):

2. Data Validation & Quality Checks – SQL

- Previewed imported data to verify structure
- Checked for missing primary identifiers
- Validated financial calculations (Net Sales logic)
- Reviewed distinct values across key business dimensions



90 %

Results | Messages

9	INV-01363	2023-10-24	Home Care	HO-041	Mashhad	Pharmacy	59320.5200	2	0.5400	118641.0000	47492.1200	23657.0000	Retail	Card	No	10	2023	118641.0000	641.000
10	INV-01364	2023-10-24	Beverages	BE-039	Ahvaz	Supermarket	50646.2900	7	15.3700	354524.0000	33685.8600	118723.0000	Retail	Transfer	Yes	10	2023	354524.0000	54490.0

	Invoice_ID	date	category	sku	region	channel	price	quantity	discount	revenue	Cost_Per_Unit	Profit	Customer_Type	Payment_Method	Promotion	Month	Year	Gross_Sales	Discount_Value	Calculated_Net_Sales

	Gross_Sales	Discount_Value	Calculated_Net_Sales	Expected_Net
3	1257532.0000	67907.0000	1189626.0000	1189625.0000
4	5912564.0000	144858.0000	5767707.0000	5767706.0000
5	629647.0000	693.0000	628954.0000	628954.0000
6	926597.0000	122311.0000	804286.0000	804286.0000
7	685949.0000	7202.0000	678747.0000	678747.0000
8	405759.0000	10225.0000	395534.0000	395534.0000
9	118641.0000	641.0000	118000.0000	118000.0000
10	354524.0000	54490.0000	300034.0000	300034.0000

	Regions	Categories	Channel
1	6	5	5

CHECKING QUERIES....(ASUS\ASUS (95))* | DATA VALIDATIO

```sql
SELECT TOP 10 * FROM fmcg_sales_dataset_new;
SELECT * FROM fmcg_sales_dataset_new
WHERE Invoice_ID IS NULL;
SELECT TOP 10
    Gross_Sales,
    Discount_Value,
    Calculated_Net_Sales,
    Gross_Sales - Discount_Value AS Expected_Net
FROM fmcg_sales_dataset_new

SELECT
    COUNT(DISTINCT region) as Regions,
    COUNT(DISTINCT category) as Categories,
    COUNT(DISTINCT channel) as Channel
FROM fmcg_sales_dataset_new
```

2. SQL Stage(SQL Server):

3. Business Logic Validation & Data Modeling

- Recalculated and validated profit logic
- Cross-checked revenue and cost consistency
- Created a clean SQL view for reporting
- Prepared structured dataset for Power BI modeling

CHECKING QUERIES....(ASUS\ASUS (95))* | DATA VALIDATION.s...(ASU

```sql
SELECT *
FROM fmcg_sales_dataset_new
WHERE price < 0
    OR quantity<0
    OR revenue<0;

SELECT TOP 20
    revenue,
    Cost_Per_Unit*quantity AS Total_cost,
    profit,
    revenue - (Cost_Per_Unit*quantity) AS Expected_profit

FROM fmcg_sales_dataset_new

CREATE VIEW vw_sales_clean AS
SELECT
    Invoice_ID,
    [date],
    category,
    sku,
    region,
    channel,
    price,
    quantity,
    revenue,
    Cost_Per_Unit,
    Profit,
    Customer_Type,
    Payment_Method,
    Promotion,
    Month,
    Year,
    Gross_Sales,
    Discount_Value,
    Calculated_Net_Sales
FROM fmcg_sales_dataset_new;

SELECT * FROM vw_sales_clean
```

Results | Messages

Invoice_ID	date	category	sku	region	channel	price	quantity	discount	revenue	Cost_Per_Unit	Profit	Customer_Type	Payment_Method	Promotion	Month	Year	Gross_Sales	Discount_Value	Calculated_Net_Sales

	revenue	Total_cost	profit	Expected_profit
1	2203280.0000	1607999.2000	595281.0000	595280.8000
2	5059786.0000	4236534.1200	823252.0000	823251.8800
3	1257532.0000	879243.9700	378288.0000	378288.0300
4	5912564.0000	4339155.7800	1573409.0000	1573408.2200
5	629647.0000	426265.4000	203381.0000	203381.6000
6	926597.0000	740026.3500	186570.0000	186570.6500
7	685949.0000	453140.8000	232809.0000	232808.2000
8	405759.0000	320167.1900	85591.0000	85591.8100
9	118641.0000	94984.2400	23657.0000	23656.7600
10	354524.0000	235801.0200	118723.0000	118722.9800
11	4211824.0000	3449546.3200	762278.0000	762277.6800

2. SQL Stage(SQL Server):

4. Exploratory Data Analysis (EDA) – SQL Insights

- Computed key KPIs (Revenue, Profit, Margin) from cleaned data
- Analyzed sales performance across **Regions** & **Channels**
- Built **Monthly trend** for time-series visualization
- Evaluated **Promotion effectiveness** and profitability impact

BUSINESS ANALYSIS...(ASUS\ASUS (69)) | CHECKING QUERIES....(ASUS\ASUS (95))*

```sql
SELECT
    SUM(quantity) as Total_Unit_sold,
    SUM(revenue) as Total_Revenue,
    SUM(profit)as Total_Profit,
    ROUND(SUM(profit) * 100.0 / SUM(revenue), 2) as Profit_Margin_Percent

FROM vw_sales_clean;

SELECT
    region,
    SUM(revenue) AS Total_Revenue,
    SUM(Profit) AS Total_Profit
FROM vw_sales_clean
GROUP BY region
ORDER BY Total_Revenue DESC;

SELECT
    YEAR,
    MONTH,
    SUM(revenue) as monthly_trend
from vw_sales_clean
group by Year,Month
order by Year,Month

SELECT
    channel,
    SUM(revenue) AS Revenue,
    SUM(Profit) AS Profit,
    ROUND(SUM(Profit) * 100.0 / SUM(revenue), 2) AS Profit_Margin
FROM vw_sales_clean
GROUP BY channel
ORDER BY Revenue DESC;

SELECT
    Promotion,
    SUM(revenue) AS Revenue,
    SUM(Profit) AS Profit
FROM vw_sales_clean
GROUP BY Promotion;
```

Results | Messages

	Total_Unit_sold	Total_Revenue	Total_Profit	Profit_Margin_Percent
1	122964	9466181758.0000	2206801003.0000	23.310000

	region	Total_Revenue	Total_Profit
1	Shiraz	1782425877.0000	410541991.0000
2	Ahvaz	1672882065.0000	400294562.0000
3	Tehran	1553360295.0000	355876515.0000
4	Mashhad	1537733102.0000	353382262.0000
5	Isfahan	1473316368.0000	342615065.0000
6	Tabriz	1446464051.0000	344090608.0000

	YEAR	MONTH	monthly_trend
23	2024	11	288728642.0000
24	2024	12	259366698.0000
25	2025	1	213764034.0000
26	2025	2	178363953.0000
27	2025	3	327501439.0000
28	2025	4	281930446.0000
29	2025	5	304086765.0000

	channel	Revenue	Profit	Profit_Margin
1	Wholesale	2567949588.0000	435124540.0000	16.940000
2	Hypermarket	1950772917.0000	541544059.0000	27.760000
3	Supermarket	1878849142.0000	564261658.0000	30.030000
4	Online	1591139034.0000	369440854.0000	23.220000
5	Pharmacy	1477471077.0000	296429892.0000	20.060000

	Promotion	Revenue	Profit
1	Yes	2161378628.0000	507750451.0000
2	No	7304803130.0000	1699050552.0000

2. SQL Stage(SQL Server):

5. Product & Customer Performance Analysis

- Identified top revenue-generating SKUs
- Compared customer segment profitability
- Distinguished between high-volume and high-margin products
- Highlighted strategic products driving overall business performance

	sku	Total_Profit
1	PE-090	20419045.0000
2	DA-088	13481623.0000
3	DA-023	12000744.0000
4	DA-134	11273518.0000
5	PE-081	11269557.0000
6	PE-100	10973501.0000
7	DA-137	10941324.0000
8	DA-020	10901525.0000

Results | Messages

	sku	Revenue
1	PE-090	70871515.0000
2	PE-081	57989151.0000
3	DA-088	56954222.0000
4	DA-020	55516059.0000
5	DA-134	48355925.0000
6	DA-064	46953470.0000
7	DA-017	45134467.0000
8	PE-100	43551697.0000

	Customer_Type	Revenue	Profit
1	Corporate	7800360535.0000	1817630745.0000
2	Retail	1665821223.0000	389170258.0000

	sku	Total_Sold
4	PE-081	567
5	DA-020	533
6	DA-064	494
7	SN-079	482

ADVANCED ANALYSI...(ASUS\ASUS (53))

```sql
SELECT TOP 10
    sku,
    SUM(revenue) AS Revenue
FROM vw_sales_clean
GROUP BY sku
ORDER BY Revenue DESC;

SELECT
    Customer_Type,
    SUM(revenue) AS Revenue,
    SUM(Profit) AS Profit
FROM vw_sales_clean
GROUP BY Customer_Type;

SELECT
    sku,
    SUM(quantity) AS Total_Sold
FROM vw_sales_clean
GROUP BY sku
ORDER BY Total_Sold DESC;

SELECT TOP 10
    sku,
    SUM(Profit) AS Total_Profit
FROM vw_sales_clean
GROUP BY sku
ORDER BY Total_Profit DESC;
```

2. SQL Stage(SQL Server):

6. Advanced SQL Analysis – Trend & Ranking Insights

- Applied CTE & window functions (LAG, RANK) for dynamic trend and ranking insights
- Built month-over-month revenue comparisons to spot performance shifts
- Ranked SKUs by total revenue to highlight top performers
- Demonstrated advanced SQL analytical capability ready for BI modeling

RANK.sql - ASUS.m...r (ASUS\ASUS (72)) | ADVANCED ANALYSI...(ASUS\A

```sql
SELECT
    sku,
    SUM(revenue) AS Total_Revenue,
    RANK() OVER (ORDER BY SUM(revenue) DESC) AS Revenue_Rank
FROM vw_sales_clean
GROUP BY sku;
```

Results | Messages

	sku	Total_Revenue	Revenue_Rank
1	PE-090	70871515.0000	1
2	PE-081	57989151.0000	2
3	DA-088	56954222.0000	3
4	DA-020	55516059.0000	4
5	DA-134	48355925.0000	5
6	DA-064	46953470.0000	6
7	DA-017	45134467.0000	7
8	PE-100	43551697.0000	8
9	PE-130	43038831.0000	9
10	DA-137	42064283.0000	10
11	PE-070	41724732.0000	11
12	PE-132	41580838.0000	12
13	DA-023	41580480.0000	13
14	PE-055	40499191.0000	14
15	DA-041	40352677.0000	15
16	PE-001	39557775.0000	16

COMPARE WITH LAS...(ASUS\ASUS (90)) | RANK.sql - ASUS.m...r

```sql
WITH Monthly_Sales AS (
    SELECT
        Year,
        Month,
        SUM(revenue) AS Monthly_Revenue
    FROM vw_sales_clean
    GROUP BY Year, Month
)
SELECT
    Year,
    Month,
    Monthly_Revenue,
    LAG(Monthly_Revenue)
        OVER (ORDER BY Year, Month) AS Previous_Month
FROM Monthly_Sales;
```

Results | Messages

	Year	Month	Monthly_Revenue	Previous_Month
1	2023	1	179614163.0000	NULL
2	2023	2	277863959.0000	179614163.0000
3	2023	3	244896096.0000	277863959.0000
4	2023	4	255365503.0000	244896096.0000
5	2023	5	290794295.0000	255365503.0000
6	2023	6	248664622.0000	290794295.0000
7	2023	7	248064671.0000	248664622.0000
8	2023	8	219021069.0000	248064671.0000
9	2023	9	293784562.0000	219021069.0000
10	2023	10	246444207.0000	293784562.0000
11	2023	11	298639050.0000	246444207.0000
12	2023	12	340143261.0000	298639050.0000
13	2024	1	259496046.0000	340143261.0000
14	2024	2	202193465.0000	259496046.0000
15	2024	3	274000088.0000	202193465.0000
16	2024	4	279034942.0000	274000088.0000

2. SQL Stage(SQL Server):

7. Advanced SQL Analysis – Market Share & Structured Aggregations

- Computed regional market share using window functions
- Applied proportional revenue contribution analysis
- Built structured monthly aggregation using CTE
- Demonstrated advanced SQL modeling and analytical thinking

MONTHLY SALES.sql...(ASUS\ASUS (87))

```sql
WITH Monthly_Sales AS (
    SELECT
        Year,
        Month,
        SUM(revenue) AS Monthly_Revenue
    FROM vw_sales_clean
    GROUP BY Year, Month
)
SELECT *
FROM Monthly_Sales;
```

Results | Messages

	Year	Month	Monthly_Revenue
1	2024	2	202193465.0000
2	2023	9	293784562.0000
3	2024	10	262948772.0000
4	2025	6	226042770.0000
5	2023	12	340143261.0000
6	2023	6	248664622.0000
7	2025	3	327501439.0000
8	2025	9	216731083.0000
9	2024	8	315339787.0000
10	2023	1	179614163.0000
11	2023	7	248064671.0000
12	2025	4	281930446.0000
13	2025	12	258505575.0000
14	2023	10	246444207.0000
15	2024	5	266218048.0000
16	2024	11	288728642.0000
17	2023	4	255365503.0000

REGION BY SALE.sq...r (ASUS\ASUS (74)) | MONTHLY SALES.s

```sql
SELECT
    region,
    SUM(revenue) AS Revenue,
    SUM(revenue) * 100.0
        / SUM(SUM(revenue)) OVER() AS Market_Share
FROM vw_sales_clean
GROUP BY region;
```

Results | Messages

	region	Revenue	Market_Share
1	Ahvaz	1672882065.0000	17.672194
2	Mashhad	1537733102.0000	16.244491
3	Shiraz	1782425877.0000	18.829406
4	Isfahan	1473316368.0000	15.563998
5	Tabriz	1446464051.0000	15.280332
6	Tehran	1553360295.0000	16.409576

3. PYTHON Stage:

- **Loaded and explored the FMCG sales dataset** using pandas, numpy, matplotlib, and seaborn.
- **Checked data structure and quality** with .shape, .head(), and .info().
- **Generated summary statistics** using .describe() to understand distribution, spread, and possible outliers.
- **Prepared the data for further cleaning and analysis** before building dashboards in Power BI.

[2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("C:\\temp\\fmcg.csv")

df.shape
df.head()
df.info()
df.describe()
# Dataset contains transactional retail sales data including sales amount, profit, discount, region, channel and product information.
```

[2]:

	price	quantity	discount	revenue	Cost_Per_Unit	Profit	Month	Year	Gross_Sales	Discount_Value	Calculated_Net_S
count	5000.000000	5000.000000	5000.000000	5.000000e+03	5000.000000	5.000000e+03	5000.000000	5000.000000	5.000000e+03	5.000000e+03	5.000000e
mean	77306.181770	24.592800	5.431820	1.893236e+06	59229.560640	4.413602e+05	6.534400	2023.995000	1.893236e+06	1.098694e+05	1.783367e
std	23442.341173	33.675484	5.204569	2.761925e+06	18593.476001	6.606657e+05	3.406162	0.818235	2.761925e+06	2.554563e+05	2.613077e
min	30452.590000	1.000000	0.000000	3.839800e+04	20353.710000	6.117000e+03	1.000000	2023.000000	3.839800e+04	0.000000e+00	3.457000e
25%	58254.730000	4.000000	2.060000	3.134500e+05	44100.685000	6.814150e+04	4.000000	2023.000000	3.134500e+05	8.227000e+03	2.956172e
50%	69936.560000	9.000000	4.000000	6.329290e+05	54128.840000	1.475095e+05	7.000000	2024.000000	6.329290e+05	2.446500e+04	6.039620e
75%	97982.732500	32.000000	5.840000	2.176386e+06	73841.587500	4.672468e+05	10.000000	2025.000000	2.176386e+06	8.261000e+04	2.030070e
max	160943.780000	156.000000	24.940000	1.834655e+07	122279.900000	5.150518e+06	12.000000	2025.000000	1.834655e+07	3.484029e+06	1.736684e

[4]:
```
df['date'] = pd.to_datetime(df['date'])
```

3. PYTHON Stage:

- **Data Transformation:** Converted date column to datetime objects for time-series analysis.
- **Data Deduplication:** Removed duplicate records to ensure data integrity and accurate aggregation.
- **Null Value Assessment:** Audited missing values to determine necessary cleaning or imputation strategies.

```
[5]: df['date'] = pd.to_datetime(df['date'])

[6]: df = df.drop_duplicates()

[7]: df.isnull().sum()

[7]: Invoice_ID          0
     date                0
     category            0
     sku                 0
     region              0
     channel             0
     price               0
     quantity            0
     discount            0
     revenue             0
     Cost_Per_Unit       0
     Profit              0
     Customer_Type       0
     Payment_Method      0
     Promotion           0
     Month               0
     Year                0
     Gross_Sales         0
     Discount_Value      0
     Calculated_Net_Sales    0
     dtype: int64
```

3. PYTHON Stage:

- **Temporal Feature Extraction:** Extracted month and year attributes to enable time-based trend analysis and seasonality comparisons.
- **KPI Creation:** Calculated profit_margin as a derived metric to quantify operational efficiency and profitability trends.

```
[15]: df['month'] = df['date'].dt.to_period('M')
      df['year'] = df['date'].dt.year
      df['profit_margin'] = df['Profit'] / df['revenue']
```

```
[17]: df.head()
```

[17]:

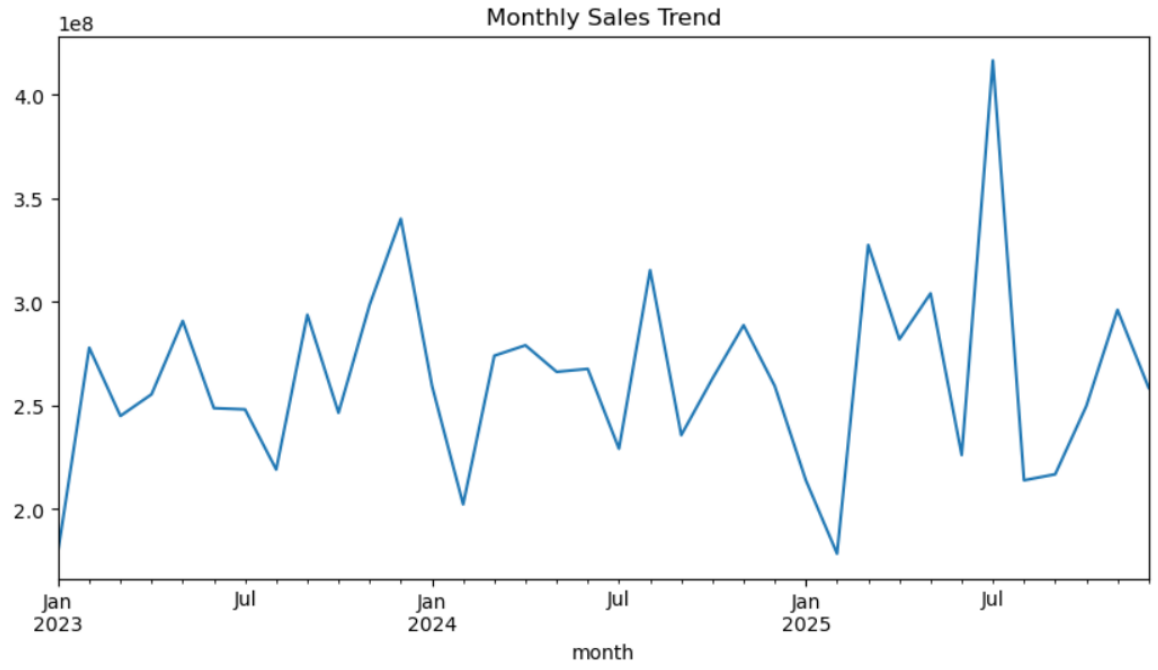
price	quantity	discount	revenue	...	Payment_Method	Promotion	Month	Year	Gross_Sales	Discount_Value	Calculated_Net_Sales	month	year	profit_margin
55904.04	6	4.00	335424	...	Card	No	1	2023	335424	13417	322007	2023-01	2023	0.177671
101091.20	6	14.83	606547	...	Transfer	Yes	1	2023	606547	89951	516596	2023-01	2023	0.211052
67767.29	4	2.38	271069	...	Card	No	1	2023	271069	6451	264618	2023-01	2023	0.126267
70094.95	56	1.77	3925317	...	Card	No	1	2023	3925317	69478	3855839	2023-01	2023	0.241229
104608.72	7	3.72	732261	...	Transfer	No	1	2023	732261	27240	705021	2023-01	2023	0.275854

3. PYTHON Stage:

- **Time-Series Analysis:** Performed group-based aggregation to visualize monthly_sales trends over time.
- **Trend Visualization:** Identified growth patterns and seasonal fluctuations, serving as a foundational input for business forecasting and decision-making.

```
[19]: monthly_sales = df.groupby('month')['revenue'].sum()

      monthly_sales.plot(figsize=(10,5))
      plt.title("Monthly Sales Trend")
      plt.show()
```



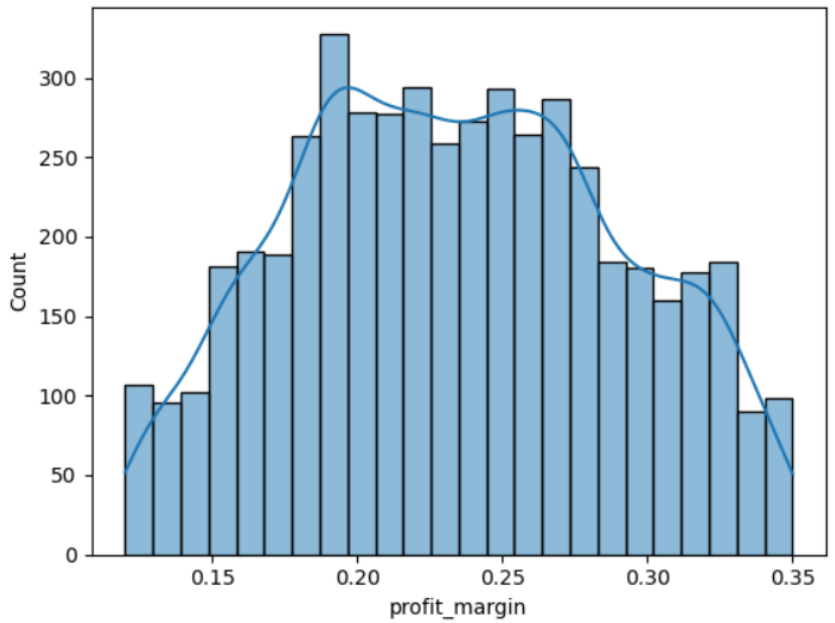
3. PYTHON Stage:

•**Performance Analytics:** Calculated Month-over-Month (MoM) growth rates to quantify business momentum and volatility.

•**Statistical Profiling:** Used histogram and KDE plots to analyze the distribution of profit_margin, identifying outliers and assessing profitability consistency.

```
[25]: sns.histplot(df['profit_margin'], kde=True)
```

```
[25]: <Axes: xlabel='profit_margin', ylabel='Count'>
```




```
[23]: mom_growth = monthly_sales.pct_change() * 100
```

```
[24]: mom_growth
```

```
[24]: month
2023-01          NaN
2023-02    54.700473
2023-03   -11.864750
2023-04     4.275040
2023-05    13.873758
2023-06   -14.487792
2023-07    -0.241269
2023-08   -11.708077
2023-09    34.135297
2023-10   -16.113970
2023-11    21.179172
2023-12    13.897784
2024-01   -23.709779
```

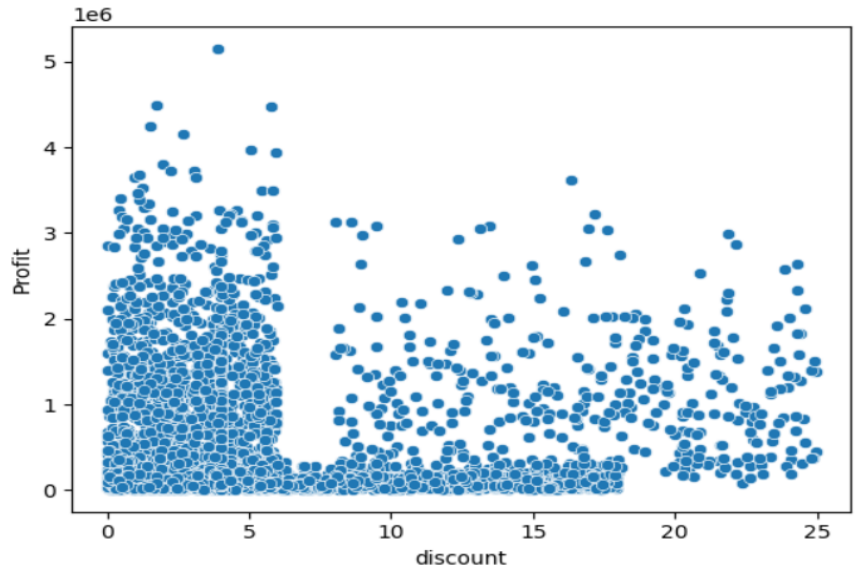
3. PYTHON Stage:

- **Discount vs Profit Analysis:** Used a scatter plot to examine the relationship between discount and Profit, revealing that higher discount levels tend to compress profitability.
- **Top-Selling Product Identification:**

Aggregated revenue by sku and ranked products to identify the top 10 revenue-generating items, supporting product performance analysis and inventory prioritization.

```
[28]: sns.scatterplot(x='discount', y='Profit', data=df)
      #Higher discount levels reduce profitability.
```

```
[28]: <Axes: xlabel='discount', ylabel='Profit'>
```




```
[30]: top_products = df.groupby('sku')['revenue'].sum().sort_values(ascending=False).head(10)
```

```
[31]: top_products
```

```
[31]: sku
      PE-090    70871515
      PE-081    57989151
      DA-088    56954222
      DA-020    55516059
      DA-134    48355925
      DA-064    46953470
      DA-017    45134467
      PE-100    43551697
      PE-130    43038831
      DA-137    42064283
      Name: revenue, dtype: int64
```

3. PYTHON Stage:

- **Pareto Principle (80/20 Rule) Application:** Performed a concentration analysis to determine which percentage of products contribute to 80% of total revenue.
- **Strategic Insights:** Identified key product drivers, enabling resource optimization and improved inventory focus.

```
[35]: n_products = len(cumulative)
pareto_threshold = cumulative[cumulative <= 0.8].shape[0]  # تا جایی که به ۸۰٪ فروش برسیم
share = pareto_threshold / n_products * 100

print(f"{share:.1f}% از محصولات، ۸۰٪ فروش را ساخته‌اند.")
```

```
53.1% از محصولات، ۸۰٪ فروش را ساخته‌اند.
```

```
[36]: region_sales = df.groupby('region')['revenue'].sum()
channel_sales = df.groupby('channel')['revenue'].sum()
```

```
[37]: region_sales
```

```
[37]: region
Ahvaz      1672882065
Isfahan    1473316368
Mashhad    1537733102
Shiraz     1782425877
Tabriz     1446464051
Tehran     1553360295
Name: revenue, dtype: int64
```

```
[33]: product_sales = df.groupby('sku')['revenue'].sum().sort_values(ascending=False)
cumulative = product_sales.cumsum() / product_sales.sum()
```

```
[34]: cumulative
```

```
[34]: sku
PE-090    0.007487
PE-081    0.013613
DA-088    0.019629
DA-020    0.025494
DA-134    0.030602
             ...
HO-127    0.999917
DA-059    0.999951
PE-140    0.999973
DA-089    0.999994
HO-075    1.000000
Name: revenue, Length: 750, dtype: float64
```

3. PYTHON Stage:

- **Regional Revenue Analysis:**

Aggregated revenue by region to compare performance across geographic markets and identify high-earning regions.

- **Channel Performance Analysis:**

Aggregated revenue by channel to evaluate which sales channels contribute most to total revenue and support distribution strategy decisions.

```
[36]: region_sales = df.groupby('region')['revenue'].sum()
      channel_sales = df.groupby('channel')['revenue'].sum()

[37]: region_sales

[37]: region
      Ahvaz       1672882065
      Isfahan     1473316368
      Mashhad     1537733102
      Shiraz      1782425877
      Tabriz      1446464051
      Tehran      1553360295
      Name: revenue, dtype: int64

[38]: channel_sales

[38]: channel
      Hypermarket     1950772917
      Online          1591139034
      Pharmacy        1477471077
      Supermarket     1878849142
      Wholesale       2567949588
      Name: revenue, dtype: int64
```

3. PYTHON Stage:

- **Multivariate Correlation Analysis:**

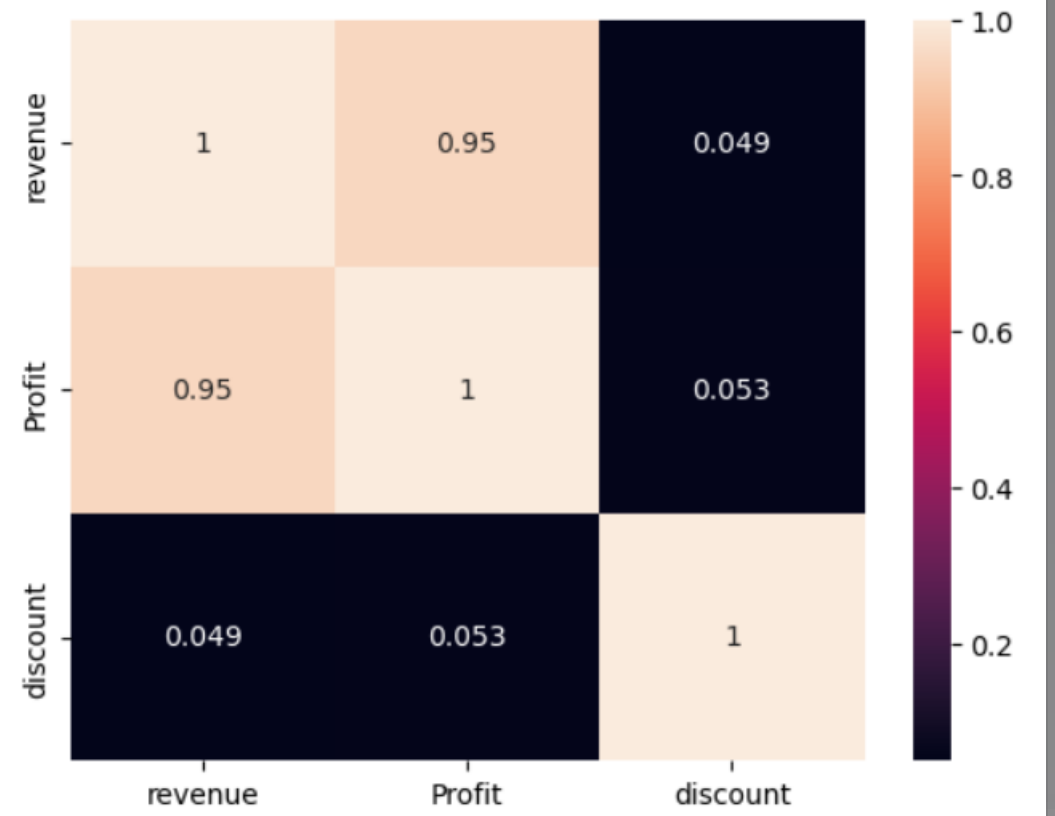
Utilized correlation matrix and heatmaps to investigate relationships between key business metrics (revenue, profit, discount).

- **Data-Driven Discovery:**

Identified the strength of correlation between revenue and profit margins, providing insights for pricing and discount policy optimization.

```
[39]: sns.heatmap(df[['revenue','Profit','discount']].corr(), annot=True)
```

```
[39]: <Axes: >
```



3. PYTHON Stage:

- **Rolling Average Analysis:** Smoothed monthly revenue trends using a 3-month moving average.
- **Pivot-Based Segmentation:** Compared monthly sales performance across product categories using a pivot table.

```
[42]: pd.pivot_table(df,
                     values='revenue',
                     index='month',
                     columns='category',
                     aggfunc='sum')
```

[42]:

category	Beverages	Dairy	Home Care	Personal Care	Snacks
month					
2023-01	48028618	39671951	37662798	43666321	10584475
2023-02	25596813	105452560	28606961	73434958	44772667
2023-03	46299740	62354794	45387488	48107622	42746452
2023-04	35381442	48550325	36818172	60969751	73645813
2023-05	23803322	73225703	44312416	73138163	76314691
2023-06	49649159	90955336	21208102	53299249	33552776
2023-07	36711004	88912517	10284518	69401020	42755612
2023-08	45218434	54729380	20823742	58053387	40196126
2023-09	63710042	64123227	53054741	69249956	43646596
2023-10	76483023	48227333	32355090	50007982	39370779
2023-11	56510399	118032548	42350491	45429514	36316098
2023-12	41032064	85398898	36194856	111991898	65525545

```
[40]: monthly_sales_rolling = monthly_sales.rolling(3).mean()
```

```
[41]: monthly_sales_rolling
```

[41]:

```
month
2023-01             NaN
2023-02             NaN
2023-03    2.341247e+08
2023-04    2.593752e+08
2023-05    2.636853e+08
2023-06    2.649415e+08
2023-07    2.625079e+08
2023-08    2.385835e+08
2023-09    2.536234e+08
2023-10    2.530833e+08
2023-11    2.796226e+08
2023-12    2.950755e+08
2024-01    2.994261e+08
2024-02    2.672776e+08
2024-03    2.452299e+08
2024-04    2.517428e+08
```

3. PYTHON Stage:

•**Synthetic Data Generation:** Implemented random sampling to create categorical variables (direction, region), simulating realistic regional and spatial data distributions.

•**Data Augmentation:** Enhanced existing datasets with additional dimensions to enable multi-faceted geographical and directional business analysis.

```
[52]: df['dir'] = np.random.choice(['North','South','West'], len(df))

[54]: df['region'] = np.random.choice(['Shiraz','Tehran','Mashhad','Ahvaz','Tabriz','Isfahan'], len(df))

[55]: df
```

[55]:

	Invoice_ID	date	category	sku	region	channel	price	quantity	discount	revenue	...	Promotion	Month	Year	Gross_Sales	Discount_Value	Cal
0	INV-00001	2023-01-01	Snacks	SN-007	Ahvaz	Pharmacy	55904.04	6	4.00	335424	...	No	1	2023	335424	13417	
1	INV-00002	2023-01-01	Dairy	DA-009	Isfahan	Online	101091.20	6	14.83	606547	...	Yes	1	2023	606547	89951	
2	INV-00003	2023-01-02	Snacks	SN-100	Mashhad	Wholesale	67767.29	4	2.38	271069	...	No	1	2023	271069	6451	
3	INV-00004	2023-01-02	Beverages	BE-135	Isfahan	Pharmacy	70094.95	56	1.77	3925317	...	No	1	2023	3925317	69478	
4	INV-00005	2023-01-02	Dairy	DA-110	Ahvaz	Online	104608.72	7	3.72	732261	...	No	1	2023	732261	27240	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
4995	INV-04996	2025-12-30	Dairy	DA-056	Tehran	Online	98186.26	55	3.48	5400244	...	No	12	2025	5400244	187929	
4996	INV-04997	2025-12-31	Dairy	DA-044	Tehran	Hypermarket	87358.23	12	17.59	1048299	...	Yes	12	2025	1048299	184396	
4997	INV-04998	2025-12-31	Snacks	SN-057	Mashhad	Pharmacy	57644.96	5	2.98	288225	...	No	12	2025	288225	8589	

3. PYTHON Stage:

- **Feature Engineering:** Engineered a binary classification feature (high_revenue) based on the revenue mean threshold, enabling targeted analysis of high-performing segments.
- **Data Exporting:** Automated the pipeline by exporting the processed, enriched dataset to Excel for final dashboarding and visualization in Power BI.

```
[56]: df['high_revenue'] = np.where(df['revenue'] > df['revenue'].mean(), 1, 0)
```

```
[57]: df
```

[57]:

channel	price	quantity	discount	revenue	...	Month	Year	Gross_Sales	Discount_Value	Calculated_Net_Sales	month	year	profit_margin	dir	high_revenue
harmacy	55904.04	6	4.00	335424	...	1	2023	335424	13417	322007	2023-01	2023	0.177671	North	0
Online	101091.20	6	14.83	606547	...	1	2023	606547	89951	516596	2023-01	2023	0.211052	North	0
holesale	67767.29	4	2.38	271069	...	1	2023	271069	6451	264618	2023-01	2023	0.126267	South	0
harmacy	70094.95	56	1.77	3925317	...	1	2023	3925317	69478	3855839	2023-01	2023	0.241229	West	1
Online	104608.72	7	3.72	732261	...	1	2023	732261	27240	705021	2023-01	2023	0.275854	West	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
Online	98186.26	55	3.48	5400244	...	12	2025	5400244	187929	5212316	2025-12	2025	0.273377	West	1
ermarket	87358.23	12	17.59	1048299	...	12	2025	1048299	184396	863903	2025-12	2025	0.282087	North	0
harmacy	57644.96	5	2.98	288225	...	12	2025	288225	8589	279636	2025-12	2025	0.186903	North	0
holesale	115619.45	7	4.96	809336	...	12	2025	809336	40143	769193	2025-12	2025	0.144072	South	0
ermarket	58305.46	6	4.31	349833	...	12	2025	349833	15078	334755	2025-12	2025	0.327822	South	0

```
[43]: df.to_excel("analysis.xlsx", index=False)
```

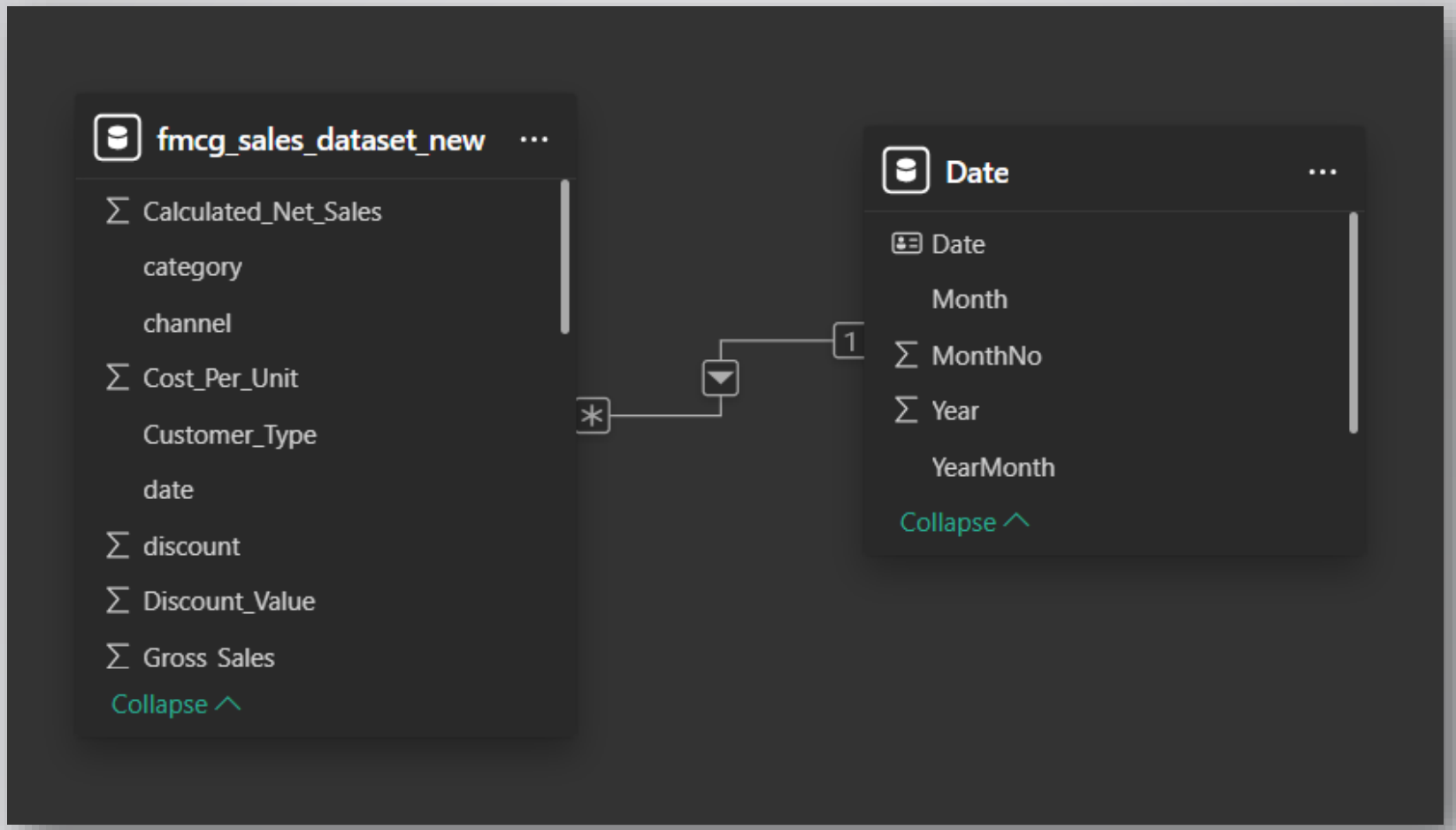
4. POWER BI Stage:

- Some calculated KPI(measures) in power bi:

KPI	DAX FORMULA
Total Orders	Total Orders = DISTINCTCOUNT(fmcg_sales_dataset_new[Invoice_ID])
Total Profit	Total Profit = SUM(fmcg_sales_dataset_new[profit])
Total sales	Total_sales = sum(fmcg_sales_dataset_new[revenue])
MoM Growth %	MoM Growth % = DIVIDE([Total_Sales] - [Previous Month Sales], [Previous Month Sales])
AOV	AOV = DIVIDE([Total_sales], [Total Orders])

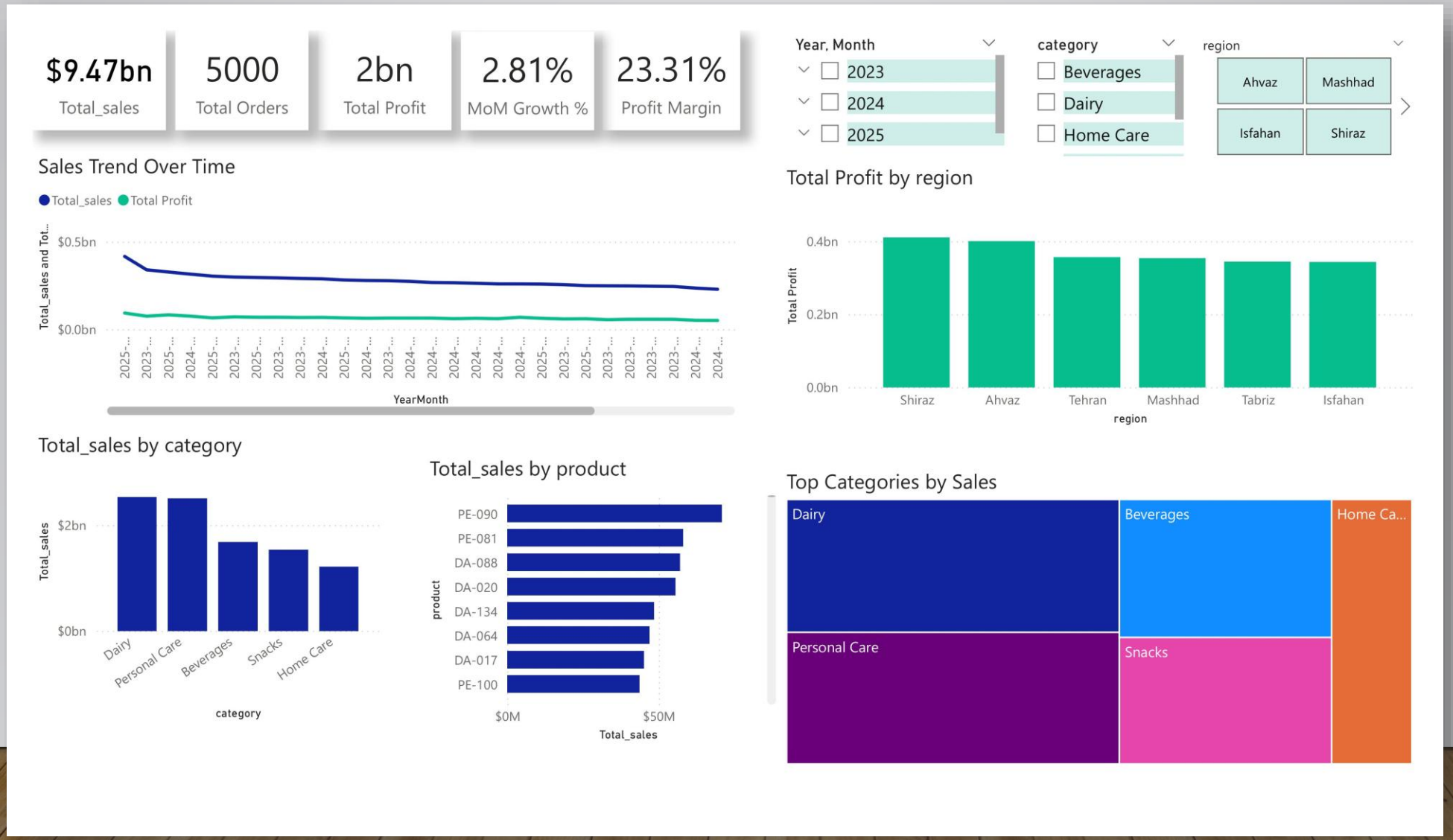
4. POWER BI Stage:

- Generate a date table based on the main database and modeling-related data.



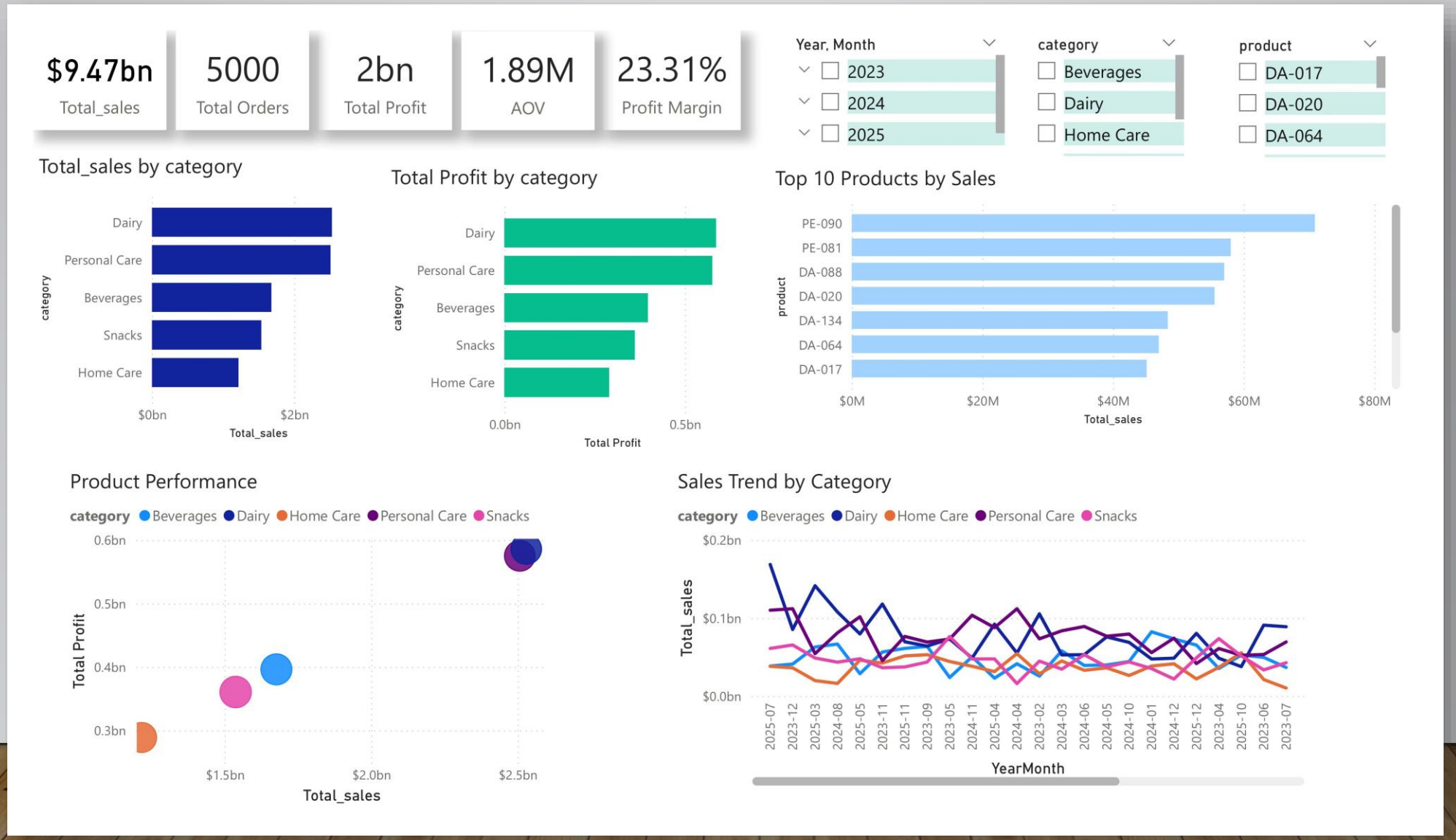
4. POWER BI Stage:

- Page 1: Overview



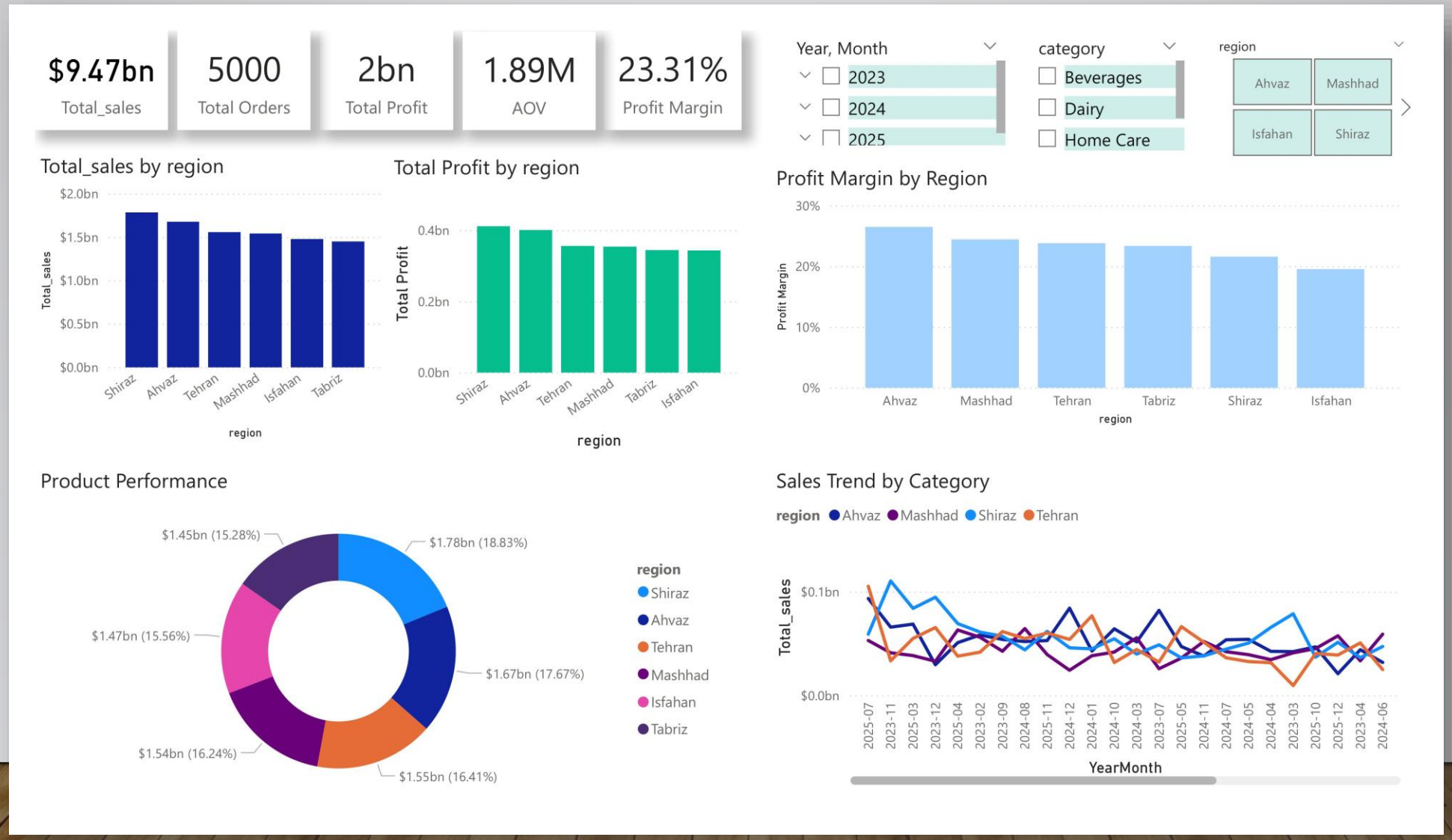
4. POWER BI Stage:

- Page 2: category analysis



4. POWER BI Stage:

- Page 3: Regional / Market Analysis

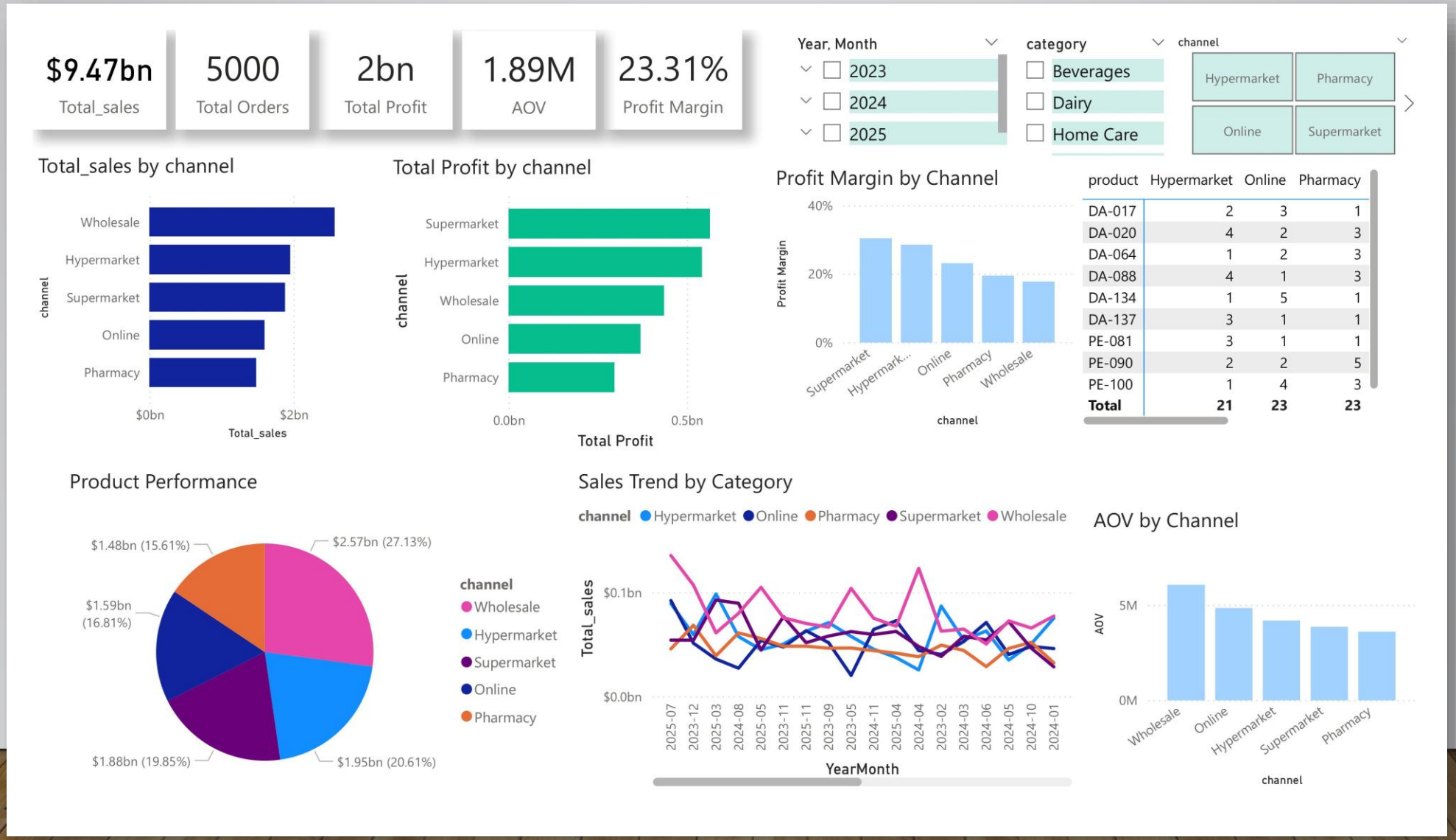


4. POWER BI Stage:

- Page 3: Time Analysis

4. POWER BI Stage:

- Page 3: Time Analysis



4. POWER BI Stage:

- Key Insights:

1. Product Portfolio Optimization (Pareto Efficiency)

•**Insight:** A dominant share of total profit is concentrated in the “Dairy” and “Personal Care” categories, reflecting an 80/20 distribution.

•**Recommendation:** Prioritize inventory management and allocate marketing resources to these high-performing segments to sustain and scale profit margins.

2. Channel & Regional Performance

•**Insight:** Significant variability exists in profit margins across sales channels (e.g., “Supermarket” dominance) and regional performance (e.g., strong sales in “Shiraz” and “Ahvaz”).

•**Recommendation:** Reallocate advertising budgets toward high-margin channels and scale regional best practices to underperforming markets to balance national revenue.

4. POWER BI Stage:

- Key Insights:

3. Growth Momentum & Market Dynamics

•**Insight:** While total sales volume is robust, MoM and YoY growth trends indicate a stabilization period, signaling a potential saturation of current market strategies.

•**Recommendation:** Deploy targeted loyalty programs and promotional campaigns to revitalize customer engagement and stimulate demand in stagnant periods.

4. Operational Efficiency & AOV Enhancement

•**Insight:** "Wholesale" shows the highest Average Order Value (AOV), whereas online and pharmacy channels face operational challenges affecting net profitability.

•**Recommendation:** Implement cross-selling and up-selling strategies within online channels to improve AOV, while optimizing logistics to mitigate operational costs.

4. POWER BI Stage:

- Key Insights:

Analysis Pillar	Key Insight	Strategic Recommendation
Product Strategy	80/20 Profitability Concentration	Focus resources on top-tier categories
Channel Strategy	Margin volatility across channels	Pivot investment to high-margin segments
Growth Trends	Plateauing MoM/YoY growth	Launch demand-generation campaigns
Regional Performance	Diverse regional potential	Scale regional success models nationally

Thank You

Data-driven insights for better decisions.